

[🏠](#) / [Мое](#) / [Мои проекты](#) / AI-агент службы поддержки

Проект: AI-агент службы поддержки

📅

15 августа
2026 г.

>_

НЕДОСТ...

📄

НЕДОСТ...

○

В ПРОЦ...

🔗

Репозиторий

🔍

Помощь

- ⚙️

Подготовка
- ✓

1
- ✓

2
- ✓

3
- >

4
- >

5
- >

6
- >

7
- >

8
- >

9
- >

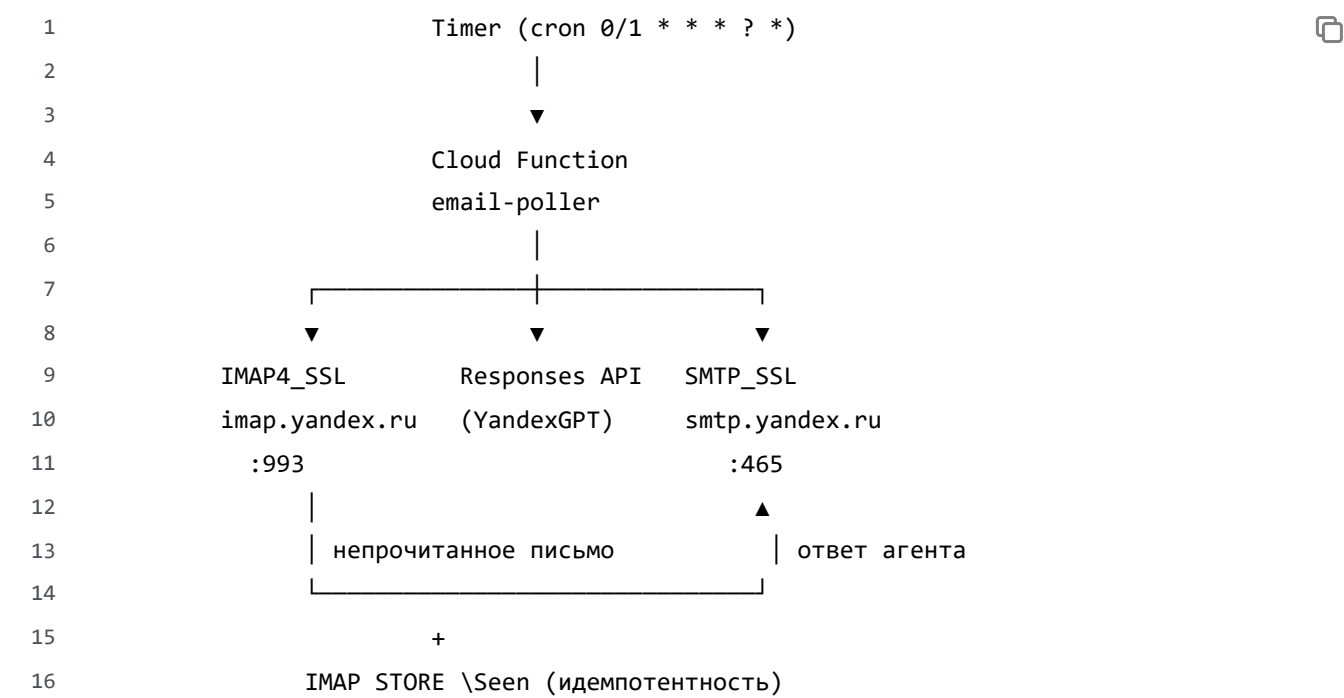
10
- 📄

Проверка

Базовый workflow по почте

Собираем минимальный рабочий цикл: сотрудник пишет на адрес helpdesk@... → письмо уходит в LLM → ответ возвращается отправщику. Пока без инструментов и без базы знаний — просто «выводим LLM в почтовый канал».

Поток данных (pull-архитектура, как и в TG-варианте):





сам забирает непрочитанное раз в минуту — без новой внешней зависимости.

Egress IMAP/SMTP из YC Cloud Functions работает — сырые TLS-сокеты (993/465) не блокируются. Это отличается от известного ограничения YC CF: исходящий POST к `api.telegram.org` режется. SMTP — это не HTTP, и блокировки для него нет.

Ссылки

- [Яндекс ID: пароли приложений](#) — для авторизации по IMAP/SMTP
- [Миграция с Assistant API на Responses API](#) — документация Yandex AI Studio
- [Таймер, который вызывает Cloud Function](#) — инструкция от Яндекса
- [imaplib — IMAP4 protocol client](#) / [smtplib — SMTP protocol client](#) — стандартная библиотека Python (для других языков возьмите их IMAP/SMTP-эквиваленты)

Задачи

1. **Создайте app-password в Яндексе** (или другом провайдере). У Яндекса: `id.yandex.ru` → Безопасность → Создать пароль приложения → Почта. Этот пароль (=не пароль аккаунта) используется и для IMAP, и для SMTP.
2. **Положите креды в Lockbox** — секрет `email-credentials` с ключом `password`. Для `IMAP_USER` / `SMTP_USER` / `HELPDESK_MAILBOX` / `OPERATOR_EMAIL` — обычные env vars, они не секрет.
3. **Напишите Cloud Function** `email-poller` на выбранном вами языке; примеры ниже — для Python-варианта (`src/email_poller.py`):
 - Раз в минуту запускается по timer-триггеру.
 - Подключается к IMAP (`imaplib.IMAP4_SSL(IMAP_HOST, 993)`), логинится, выбирает INBOX .
 - Забирает непрочитанные письма: `imap.search(None, "UNSEEN")` .
 - Для каждого письма: достаёт `From:` (`email.utils.parseaddr`), текст (`msg.get_body(preferencelist=("plain",))`), вызывает Responses API (`https://rest-assistant.api.cloud.yandex.net/v1/responses`).
 - Отправляет ответ через SMTP (`smtplib.SMTP_SSL(SMTP_HOST, 465) + send_message`).
 - Маркирует исходное письмо `\Seen` (`imap.store(num, "+FLAGS", "\\Seen")`) — иначе при следующем запуске оно снова попадёт в выборку.
 - С шага про YDB poller также пишет историю переписки в YDB и передаёт её агенту как контекст (multi-turn) — тогда в деплой добавятся `ydb` в requirements и YDB-секреты, см. шаг про MCP и YDB.



Все курсы

язык поменяйте `--runtime`, `--entrypoint` и `--source-path` (доступные варианты — в документации Cloud Functions):

```
1  yc serverless function version create \
2    --function-name email-poller \
3    --runtime python312 \
4    --entrypoint email_poller.handle \
5    --memory 256m \
6    --execution-timeout 120s \
7    --source-path src/email_poller.py \
8    --service-account-id <SA_ID> \
9    --environment YC_FOLDER_ID=<folder-id>,IMAP_HOST=imap.yandex.ru,IMAP_USER=helpdesk@y
10   --secret environment-variable=IMAP_PASSWORD,name=email-credentials,key=password \
11   --secret environment-variable=SMTP_PASSWORD,name=email-credentials,key=password
```

После шага про YDB (poller пишет историю в `messages`) деплой меняется: зависимости и YDB-переменные. Упаковка — `zip` (как у `ydb-tickets`): `zip -j /tmp/email-poller.zip src/email_poller.py && cp src/requirements-poller.txt /tmp/requirements.txt && zip -j /tmp/email-poller.zip /tmp/requirements.txt` — YC CF ищет в корне архива файл с именем `requirements.txt`. В команду деплоя добавляются `--source-path /tmp/email-poller.zip`, `--secret environment-variable=YDB_ENDPOINT,name=ydb-endpoint,key=value` `--secret environment-variable=YDB_DATABASE,name=ydb-database,key=value`, а SA нужна роль `ydb.editor` на каталог.

5. Создайте timer-триггер:

```
1  yc serverless trigger create timer \
2    --name email-poller-trigger \
3    --cron-expression "0/1 * * * ? *" \
4    --invoke-function-name email-poller \
5    --invoke-function-tag '$latest' \
6    --invoke-function-service-account-id <SA_ID>
```

Cron-формат — AWS EventBridge (`0/1 * * * ? *` = каждую минуту).

6. **Ручной тест:** отправьте письмо на `helpdesk@yandex.ru` с текстом «привет», дождитесь ответного письма (в пределах одного цикла таймера — до 60 секунд), или `yc serverless function invoke email-poller` — должно вернуть `{"processed": N, ...}`.

Подсказки



Все курсы

`functions.functionInvoker` у него уже есть. не забудьте выдать ему `lockbox.payloadViewer` на секрет `email-credentials`.

- Каждое обработанное письмо обязательно маркируйте `\Seen`. Альтернатива (надёжнее при ошибках в середине цикла) — сохранять последний UID в YDB, но для MVP достаточно флага.
- Если письмо `multipart` — `msg.get_body(preferencelist=('plain',))` выбирает `text/plain` часть. Для писем только в HTML нужен fallback на `html.parser` или `beautifulsoup` (последний — внешний пакет).
- У Яндекса IMAP и SMTP используют одни и те же `user + app-password`. Поэтому `--secret` для `IMAP_PASSWORD` и `SMTP_PASSWORD` ссылается на один и тот же секрет `email-credentials` с ключом `password`.
- При ошибке обработки конкретного письма (например, агент недоступен) всё равно маркируйте его `\Seen` — иначе `poller` заикнется на одном проблемном письме и не дойдёт до остальных. Шаблон: `try/except + _imap_mark_seen` в ветке `except`.

Результат шага

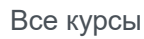
- Вы пишете письмо на `helpdesk@yandex.ru` с темой «Тест» и текстом «привет» — в течение минуты (один цикл таймера) приходит ответ от YandexGPT с темой `Re: Тест`.
- В логах CF `email-poller` видно цепочку: `GOT_UNSEEN=1 → MSG num=... from=... subject=... → AGENT_OK len=... → SEND_OK to=...`
- Исходное письмо в ящике помечено как прочитанное.

Приложение: Вариант Telegram

Если предпочитаете Telegram — соберите `telegram-poller` по той же `pull`-схеме (CF раз в минуту забирает `getUpdates` и шлёт ответ через `sendMessage` GET). Идея та же, что у почтового поллера, — меняется только транспорт.

Отличия от почтового варианта:

- Telegram требует создания бота через `@BotFather`, кред — один токен (`TELEGRAM_BOT_TOKEN`).
- Идемпотентность — сдвигом `offset` в `getUpdates?offset=<update_id+1>`, а не флагом `\Seen`.
- Известная особенность YC CF: исходящий POST к `api.telegram.org` блокируется, поэтому `sendMessage` переписан на GET с `query params`. IMAP/SMTP такой проблемы нет.



Переменные окружения для TG-варианта — в закомментированном блоке `code/.env.example`.

Завершить шаг

A screenshot of a web browser's address bar and toolbar. The address bar shows the URL "http://localhost:3000/". The toolbar contains various icons for text formatting (bold, italic, underline, text color, background color), a link icon, an image icon, a list icon, a grid icon, a search icon, and a refresh icon.

 Отправить

Комментарии

Альгис Швиторайтис

18 сентября 2026 г., 17:14

Секреты добавлены в Lockbox Сервисный аккаунт создан, права настроены Аген help-desk создан Readme заполнен

Хекслет

О нас

Отзывы

Справка

Блог

Вопросы и ответы

Глоссарий

Карта сайта

Направления

Программирование

III

Аналитика

DevOps

Тестирование

Фронтенд

Бэкенд